LiDAR Drivers and SLAM Repos: Verified Facts (Mid-360 + Airy, Humble, Orin)

Verified 2026-10-18 from raw repo files and the GitHub API.

Livox-SDK2 + livox_ros_driver2 (Mid-360)

- **Repos:** `Livox-SDK/Livox-SDK2` (master) and `Livox-SDK/livox_ros_driver2` (master), both MIT. The SDK installs to /usr/local.
- **`build.sh` danger:** `./build.sh humble` runs `rm -rf ../../build ../../devel ../../install` and wipes the workspace. Build by hand instead:
  1. `cp package_ROS2.xml package.xml`
  2. `cp -r launch_ROS2 launch`
  3. `colcon build --cmake-args -DROS_EDITION=ROS2 -DDISTRO_ROS=humble`
- **Topics:** `livox/lidar`, plus `livox/imu` as `sensor_msgs/Imu` with frame `livox_frame` (hard-coded). **IMU acceleration is in g.**
- **`xfer_format`:**
  - `0` = PointCloud2 with fields x, y, z, intensity (f32), tag (u8), line (u8), **timestamp (f64, absolute ns)**.
  - `1` = `livox_ros_driver2/CustomMsg` with timebase (u64 ns) and per-point `offset_time` (u32 ns).
- **`MID360_config.json`:** host IP 192.168.1.5, ports 56100-56500 / 56101-56501. LiDAR IP is **192.168.1.1XX, where XX is the last two digits of the serial**.
- **Known issues:** frame drops on Orin (#261), CycloneDDS issues (#73).

rslidar_sdk (Airy)

- **Repos:** `RoboSense-LiDAR/rslidar_sdk` v1.5.20 (main, with submodule `rs_driver`) and `RoboSense-LiDAR/rslidar_msg` (master). BSD-3.
- **CMake changes:**
  - Set `POINT_TYPE XYZIRT` (default is XYZI).
  - Set `ENABLE_IMU_DATA_PARSE ON` (default OFF).
  - Do a clean rebuild after changing either, or the IMU won't appear (#200).
- **`config.yaml`:**
  - `lidar_type: RSAIRY`, `msop_port: 6699`, `difop_port: 7788`, **`imu_port: 6688`**.
  - `use_lidar_clock: true`, `ts_first_point: true`.
  - Topics `/rslidar_points` and `/rslidar_imu_data`, frame `rslidar`.
  - Default LiDAR IP 192.168.1.200; PC 192.168.1.102.

- **Point fields:** x, y, z, intensity (f32), ring (u16), **timestamp (f64, absolute seconds)**.
- **IMU:**
  - Acceleration is **in g**, gyro in rad/s.
  - It shares frame_id `rslidar` with the cloud but is **physically rotated**: `R = [0,-1,0; -1,0,0; 0,0,-1]`, T = 0 (issue #203, KiiiLin fork).
- **Humble:** use CycloneDDS (`launch/humble_start.py`). The Airy may publish below 10 Hz on Jetson (#258).

FAST-LIO (ROS 2)

- **Repo:** `hku-mars/FAST_LIO`, branch **`ROS2`**, with recursive submodule ikd-Tree. GPL-2.0.
- **It includes `livox_ros_driver2/msg/custom_msg.hpp`,** so the Livox driver must be built first, even for an Airy-only build.
- **`lidar_type` values:** 1 = Livox CustomMsg, 2 = Velodyne (`time`, `ring`), 3 = Ouster (`t`), **4 = MID360 PointCloud2**.
  - **Avoid type 4.** It ignores per-point time and reads `reflectivity`, which doesn't exist in the PointCloud2.
  - **For the Mid-360 use `lidar_type: 1` with `xfer_format: 1`.**
- **`mid360.yaml`:** `extrinsic_T [-0.011, -0.02329, 0.04412]`, R = I, blind 0.5, `timestamp_unit` 3.
- **Outputs:** `/cloud_registered`, `/cloud_registered_body`, `/Laser_map`, `/Odometry`, `/path`.
- **IMU units:** g or m/s² both work, since FAST-LIO normalises internally. The sensor must be static during initialisation.
- **Airy fork:** `KiiiLin/fast_lio_robosenseAiry` (main, ★43), package `fast_lio_robosense`, `lidar_type 5`, `robosenseAiry.yaml`.
- **Our approach:** add our own `ROBOSENSE=5` handler to FAST-LIO (point struct `{x, y, z, intensity, uint16 ring, double timestamp}`, curvature = (t − t0)·1000 ms).

GLIM

- **Repos:** `koide3/glim` v1.2.2 (MIT) and `glim_ros2`. `glim_ext` is "half-baked"; its ScanContext module is CC BY-NC-SA (**non-commercial**).
- **Jetson:** tested on Orin with JetPack 6.1. The PPA has **arm64 jammy** builds: `ros-humble-glim-ros-cuda12.6` (JP 6.1/6.2) and `-cuda12.2` (JP 6.0).

```
curl -s https://koide3.github.io/ppa/setup_ppa.sh | sudo bash
sudo apt install libiridescence-dev libboost-all-dev libglfw3-dev
libmetis-dev libgtsam-points-cuda12.6-dev ros-humble-glim-ros-cuda12.6
```

- **Input: PointCloud2 only**, with a time field named `t`, `time` or `timestamp`.
  - `autoconf_perpoint_times` handles both Livox absolute-ns f64 and RoboSense absolute-seconds f64.
  - `acc_scale` 0 means auto-detect; we set 9.80665 explicitly.
- **Configs:**
  - `config_ros.json`: topics, plus `extension_modules`. Drop `libstandard_viewer.so` to run headless.
  - `config_sensors.json`: `T_lidar_imu` as TUM `[x y z qx qy qz qw]`.
  - No Mid-360 or Airy presets ship with it.
  - CPU variant: `config_odometry_cpu` + `sub_mapping_passthrough` + `global_mapping_pose_graph`.
- **Offline:** `ros2 run glim_ros glim_rosbag <bag>`.
  - Writes **`/tmp/dump`** on exit, including `traj_lidar.txt` and `traj_imu.txt` in TUM format.
  - `offline_viewer` exports PLY.
- **Tip:** run the driver and GLIM as components in one container to avoid DDS drops.

Mid-360 format conflict

FAST-LIO/Point-LIO want CustomMsg; GLIM wants PointCloud2. **Our design:** record CustomMsg, and run `dt_livox_bridge` (CustomMsg → PointCloud2 with a per-point f64 `timestamp`) for GLIM and other consumers.

Others

- **Point-LIO ROS2:** `dfloreaa/point_lio_ros2` (Humble, includes `mid360.yaml`, `acc_norm 1.0`). Airy port at `morte2025/Point-LIO-ros2-robosense-Airy` (unverified).
- **FAST-LIVO2 ROS2:** `integralrobotics/FAST-LIVO2` branch `ros2`. Needs Sophus and the rpg_vikit ROS2 port, and expects a **hardware-triggered** camera, so it is deferred for the USB-connected ZED.
- **KISS-ICP:** `PRBonn/kiss-icp`, ROS2 wrapper in `ros/`, also `pip kiss-icp`. MIT. LiDAR-only.